csvtrim

Cleans up a CSV file: strips leading and trailing whitespace from every cell and drops rows that are completely empty. One Python file, standard library only.

```
python3 csvtrim.py sample.csv
```

`sample.csv` has padded cells, a blank line, and a `,,` row. The output:

```
name,city,age
Ana,Lisbon,31
Ben,Oslo,27
```

The count of dropped rows goes to standard error (`csvtrim: dropped 2 empty rows`), so it never ends up in redirected output.

Usage

```
python3 csvtrim.py in.csv -o out.csv
python3 csvtrim.py in.tsv --delimiter $'\t'
python3 csvtrim.py in.csv --keep-empty
```

Option	Effect
`-o`, `--output FILE`	Write to FILE instead of standard output. Overwrites an existing file.
`--delimiter D`	Field delimiter for both input and output. Default `,`.
`--keep-empty`	Still strip cells, but keep rows that are completely empty.

A missing input file prints `csvtrim: no such file: <name>` and exits with status 2.

Gotchas

- Only the ends of each cell are trimmed. Spaces inside a cell, including a line break inside a quoted cell, stay as they are.
- A row counts as empty when every cell is blank after trimming, so a row of spaces and commas is dropped.
- With `--keep-empty`, a blank line in the input is written back as a blank line.
- The input is read in full before the output is written, so `-o` with the same path as the input works. It replaces the original, so keep a copy if you need it.
- The delimiter is one setting for both reading and writing. You cannot convert between delimiters.
- Input must be UTF-8. Output is UTF-8 with `\n` line endings.

Develop

```
python3 -m unittest
```

The tests cover the `trim()` function (cell stripping, dropping empty rows, `--keep-empty`). The command-line handling has no tests.